

[Nom de la société]

Étude de Cas : Application de la méthodologie RUP

Thème : Gestion des Informations des Étudiants

Réalisé par : SOFIA MOTAI
11/04/2025

I. Analyse :

1. Introduction :

1.1 Objectif du projet :

Le projet "**Gestion des Informations des Étudiants**" a pour objectif de développer une application Java permettant de gérer les informations des étudiants. L'application doit offrir des fonctionnalités d'ajout, de suppression et de modification des données des étudiants, tout en assurant une gestion fluide et robuste des informations. Elle doit également permettre une gestion fiable des erreurs et une validation correcte des entrées.

1.2 Contexte :

Ce projet s'inscrit dans le cadre d'un exercice de développement Java. L'objectif est de comprendre les concepts de gestion de données, tout en développant une application simple et bien structurée. Le projet doit démontrer des bonnes pratiques de programmation, ainsi que l'application de tests unitaires et l'analyse de la qualité du code via des outils comme **JUnit**.

2. Analyse des besoins :

2.1 Fonctionnalités principales :

Les fonctionnalités attendues de l'application sont les suivantes :

- **Ajout d'un étudiant** : Permet à l'utilisateur d'ajouter les informations d'un nouvel étudiant (nom, prénom, ID étudiant, etc.).
- **Suppression d'un étudiant** : Permet de supprimer un étudiant de la liste en fonction de son identifiant unique.
- **Modification d'un étudiant** : Permet de modifier les informations d'un étudiant existant, comme son nom, prénom ou toute autre donnée associée à son identifiant.

2.2 Contraintes et exigences :

- **Technologie** : Le projet doit être développé en Java.

- **Testabilité** : Les fonctionnalités principales (ajout, suppression, modification) doivent être couvertes par des tests unitaires.
- **Simplicité** : Le projet doit être facile à maintenir et à étendre, tout en étant robuste et en minimisant les erreurs utilisateur.

3. Technologies utilisées :



Apache NetBeans : est un **environnement de développement intégré (IDE)** open-source, principalement utilisé pour le développement d'applications Java. Il permet aux développeurs de créer, compiler, exécuter, déboguer et tester des programmes dans un environnement unifié et convivial.



MySQL : est un **système de gestion de base de données relationnelle (SGBDR)** open-source, basé sur le langage **SQL (Structured Query Language)**. Il permet de **créer, gérer, interroger et manipuler** des bases de données de manière efficace et sécurisée.



Visual Paradigm : est un **outil de modélisation UML (Unified Modeling Language)** professionnel utilisé pour concevoir, documenter et visualiser l'architecture des systèmes logiciels. Il aide les développeurs, analystes et architectes à mieux comprendre, planifier et structurer leurs projets.

Il prend en charge plusieurs types de diagrammes, notamment :

- **Diagrammes de classes**
- **Diagrammes de cas d'utilisation**
- **Diagrammes de séquence**
- **Diagrammes d'activités**
- Et bien d'autres...

4. Exigences Fonctionnelles :

- **R1** : Authentification nécessaire pour accéder à l'application.
- **R2** : Ajouter les informations de l'étudiant

- **R3** : Afficher une liste qui contient les informations de tous les étudiants
- **R4** : Sélectionner un étudiant pour le modifier ou le supprimer définitivement
- **R5** : Modifier les informations de l'étudiant
- **R6** : Supprimer l'étudiant sélectionné

5. Exigences de Qualité :

- **R7** : L'application doit être conviviale et intuitive pour garantir une expérience utilisateur positive.
- **R8** : L'application doit être utilisable par l'utilisateur sans grandes connaissances en informatique.

6. Exigences de Performance :

- **R9** : Répondre à une requête (recherche patient) en moins de 2 secondes.
- **R10** : Gérer une base de données de plusieurs patients sans perte de performance.

<<requirement>> R1
Text = "Authentification nécessaire pour accéder à l'application" ID = "REQ001" source = "" kind = "Functional" verifyMethod = "Test" risk = "High" status = "Approved"

<<requirement>> R2
Text = "Ajouter les informations de l'étudiant" ID = "REQ002" source = "" kind = "Functional" verifyMethod = "Test" risk = "Medium" status = "Approved"

<<requirement>> R3
Text = "Afficher une liste qui contient les informations de tous les étudiants" ID = "REQ003" source = "" kind = "Functional" verifyMethod = "Test" risk = "Low" status = "Approved"

<<requirement>> R10
Text = "Gérer une base de données de plusieurs patients sans perte de performance" ID = "REQ010" source = "" kind = "Performance" verifyMethod = "Test" risk = "Medium" status = "Approved"

<<requirement>> R4
Text = "Sélectionner un étudiant pour le modifier ou le supprimer définitivement" ID = "REQ004" source = "" kind = "Functional" verifyMethod = "Test" risk = "Medium" status = "Approved"

<<requirement>> R5
Text = "Modifier les informations de l'étudiant" ID = "REQ005" source = "" kind = "Functional" verifyMethod = "Test" risk = "Medium" status = "Approved"

<<requirement>> R6
Text = "Supprimer l'étudiant sélectionné" ID = "REQ006" source = "" kind = "Functional" verifyMethod = "Test" risk = "High" status = "Approved"

<<requirement>> R7
Text = "L'application doit être conviviale et intuitive pour garantir une expérience utilisateur" ID = "REQ007" source = "" kind = "Interface" verifyMethod = "Inspection" risk = "Medium" status = "Approved"

<<requirement>> R8
Text = "L'application doit être utilisable par l'utilisateur sans grandes connaissances en informatique" ID = "REQ008" source = "" kind = "Interface" verifyMethod = "Inspection" risk = "Medium" status = "Approved"

<<requirement>> R9
Text = "Répondre à une requête (recherche patient) en moins de 2 secondes" ID = "REQ009" source = "" kind = "Performance" verifyMethod = "Test" risk = "High" status = "Approved"

II. Conception :

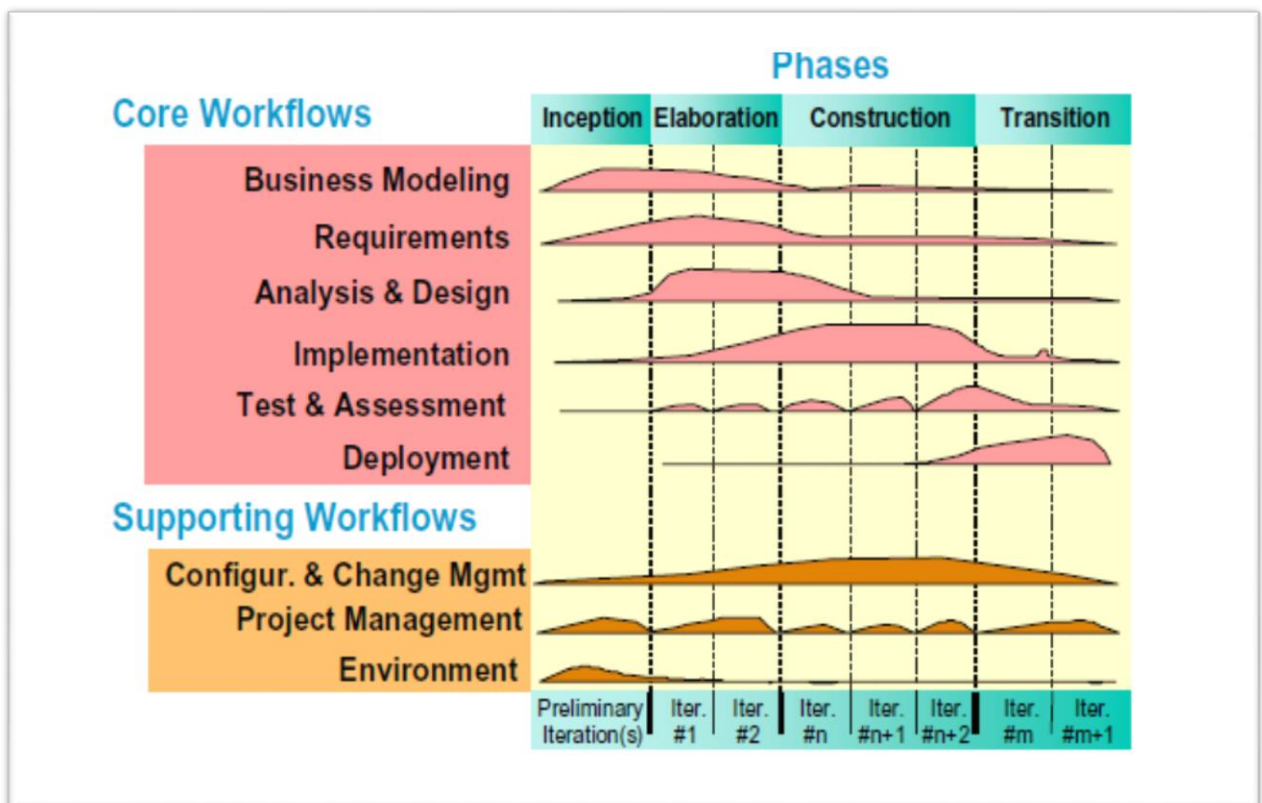
1. Méthodologie RUP :

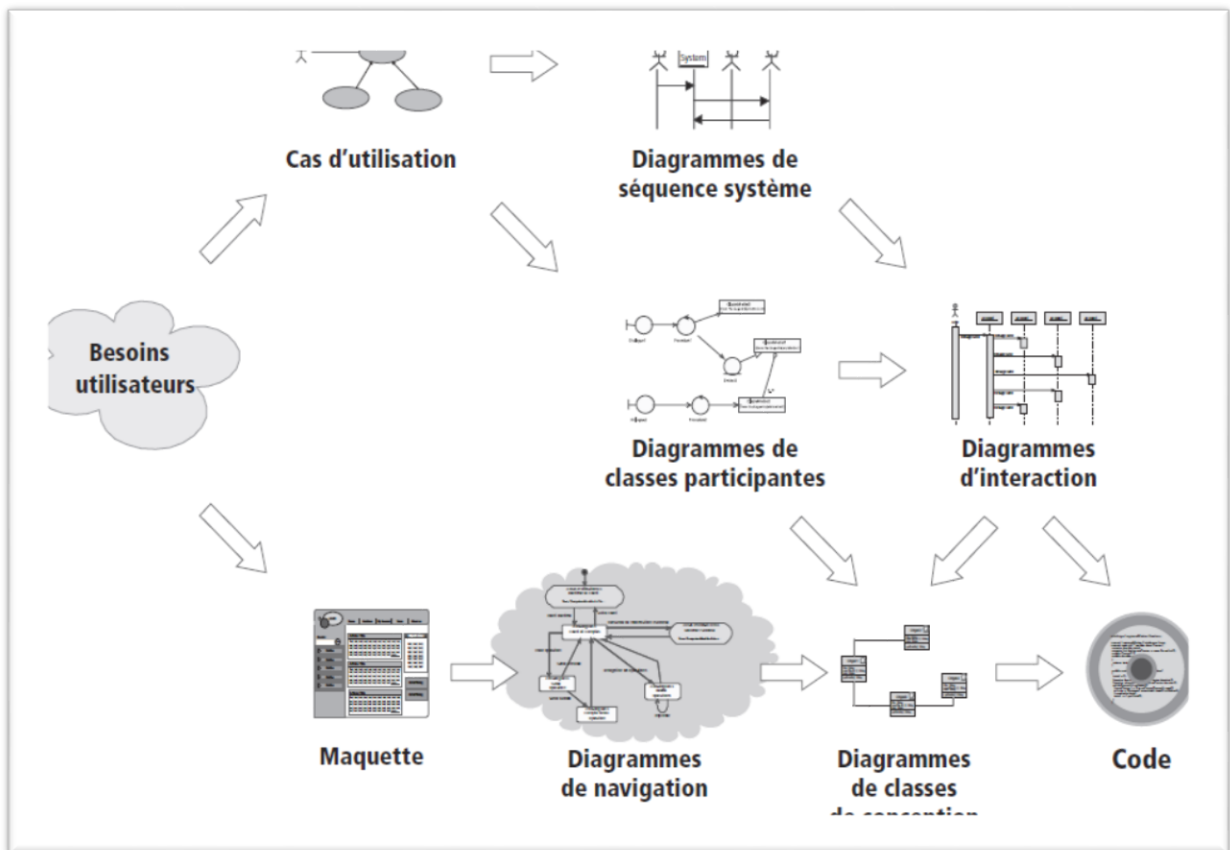
1.1 Définition :

RUP (Rational Unified Process) est une **méthodologie de développement logiciel** proposée par IBM Rational. Elle repose sur une approche **itérative, incrémentale et orientée objet**, conçue pour garantir un développement structuré, maîtrisé et de qualité.

Le processus RUP est divisé en **quatre phases principales** :

- ✓ **Inception** : définition des objectifs, étude de faisabilité, identification des exigences principales.
- ✓ **Élaboration** : analyse des besoins, modélisation du système (UML), élaboration de l'architecture.
- ✓ **Construction** : implémentation du système, codage, intégration.
- ✓ **Transition** : tests, déploiement et livraison du produit final.





1.2 Caractéristiques de la méthodologie RUP :

- ✓ Basé sur les **cas d'utilisation**.
- ✓ Repose sur l'**architecture** du logiciel.
- ✓ Encourage une **documentation claire** à chaque phase.
- ✓ Intègre les **tests** tout au long du cycle.

2. Cas d'utilisation :

2.1 Acteur principale :

L'administrateur joue un rôle central dans la gestion et le bon fonctionnement du système. Il est responsable de la supervision et de la maintenance des informations des étudiants. Ses principales responsabilités incluent :

- **Ajout d'informations :**
L'administrateur peut enregistrer de nouveaux étudiants dans le système en saisissant leurs données personnelles, académiques et administratives (nom, prénom, matricule, date de naissance, filière, etc.).
- **Modification des informations :**
Il peut mettre à jour les données existantes des étudiants en cas de

changement (ex. : changement de filière, mise à jour de contact, correction d'erreurs).

- **Suppression des informations :**

L'administrateur a l'autorité de supprimer un étudiant du système, par exemple après une fin d'études ou une annulation d'inscription.

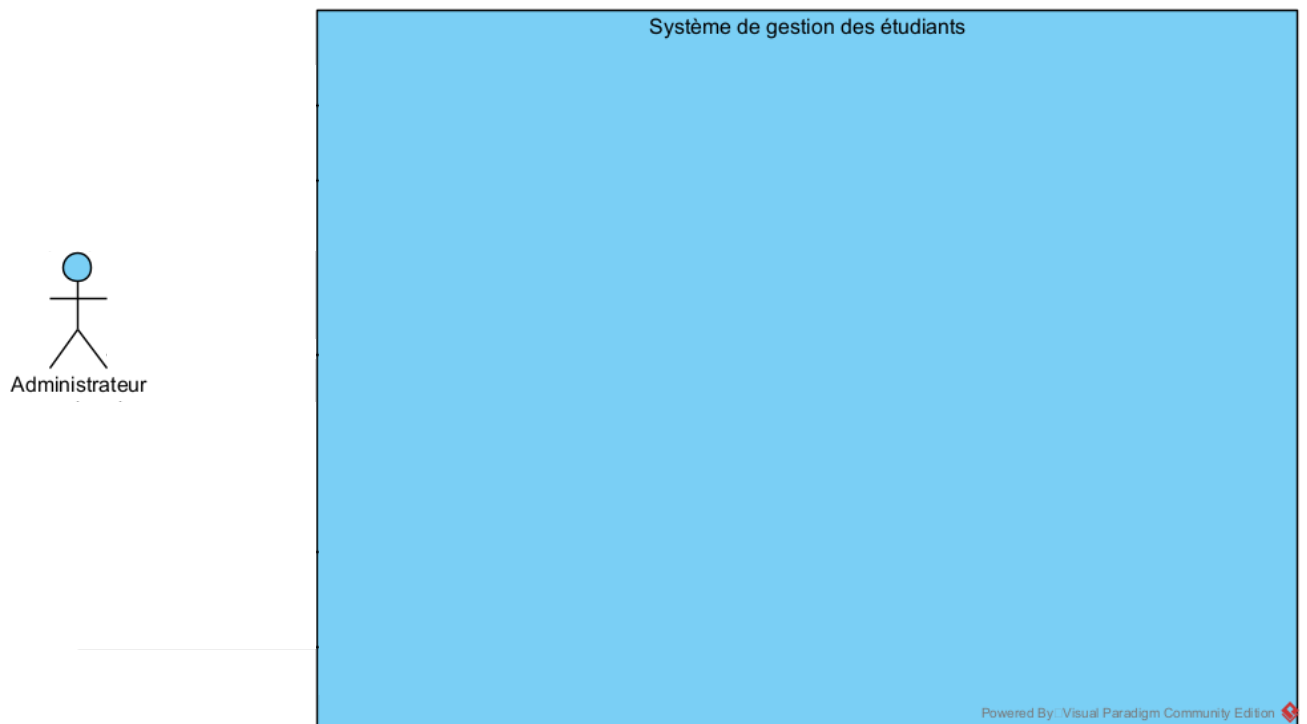


Figure : Diagramme de contexte statique

2.2 Identification des cas d'utilisation :

A partir des exigences précédentes, nous pouvons tirer les cas d'utilisation suivants :

- **UC1 :** Authentification
- **UC2 :** Ajouter étudiant
- **UC3 :** Afficher la liste des étudiants
- **UC4 :** Sélectionner un étudiant pour la modification ou la suppression
- **UC5 :** Modifier un étudiant
- **UC6 :** Supprimer un étudiant

2.3 Regroupement des exigences fonctionnelles :

Exigences Fonctionnelles	Intention d'acteur	Acteur
R1 : Authentification nécessaire pour accéder à l'application	Authentification	Administrateur
R2 : Ajouter les informations de l'étudiant	Ajouter un étudiant	Administrateur
R3 : Afficher une liste qui contient les informations de tous les étudiants	Afficher la liste des étudiants	Administrateur
R4 ; Sélectionner un étudiant pour le modifier ou le supprimer définitivement	Sélectionner un étudiant pour le modification suppression	Administrateur
R5 : Modifier les informations de l'étudiant	Modifier un étudiant	Administrateur
R6 : Supprimer l'étudiant sélectionné	Supprimer un étudiant	Administrateur

2.4 Diagramme de cas d'utilisation :

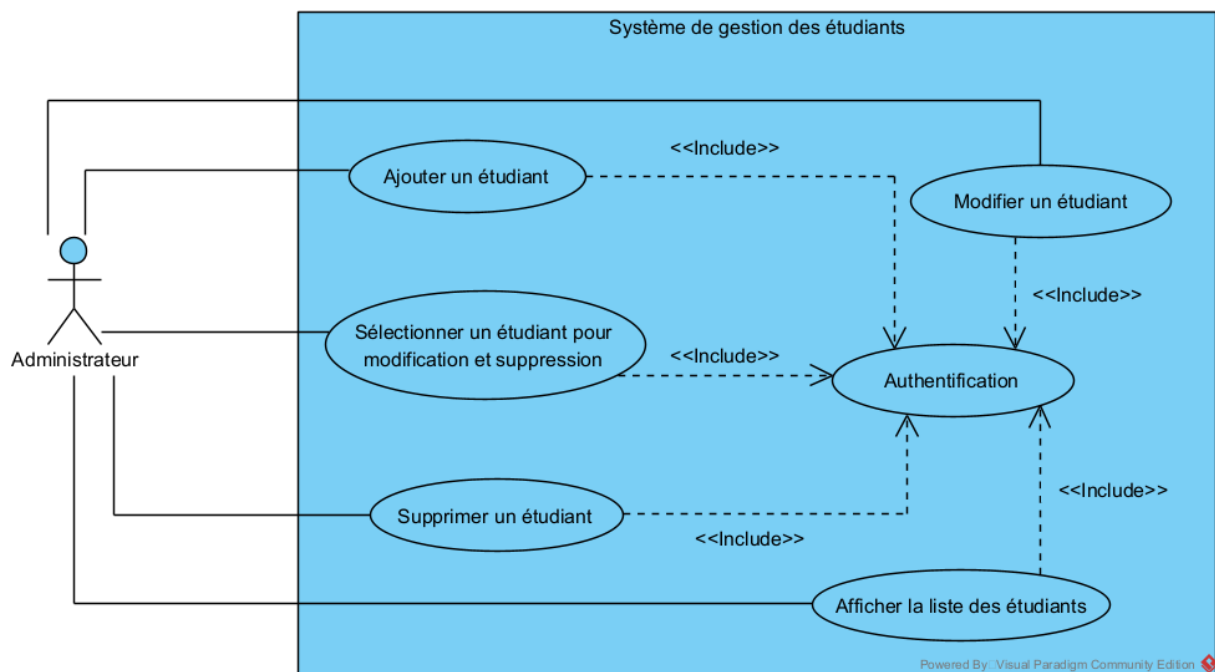


Figure : diagramme de cas d'utilisation

2.5 Classement des cas d'utilisation :

Pour classer les cas d'utilisation, il est important de décider qu'ils sont les cas les plus prioritaires et qu'ils sont les cas qui présente de grands risques techniques.

Cas d'utilisation	Priorité	Risque
• UC1 : Authentification	Haute	Haut
• UC2 : Ajouter un étudiant	Moyenne	Moyen
• UC3 : Afficher la liste des étudiants	Basse	Bas
• UC4 : Sélectionner un étudiant pour le modification suppression	Moyenne	Moyen
• UC5 : Modifier un étudiant	Moyenne	Moyen
• UC6 : Supprimer un étudiant	Haute	Haute

3. Diagrammes de séquences :

3.1 Authentification :

➤ **Flux des événements :**

1. Entrer le nom et le mot de passe
 - 1.1 Envoie le nom complet et le mot de passe
 - 1.2 Vérification des identifiants
 - 1.2.1 Si valide, redirige vers l'interface de gestion
 - 1.2.2 Si invalide, affiche message d'erreur
 - 1.2.2.1 Affiche interface de gestion des informations des étudiants

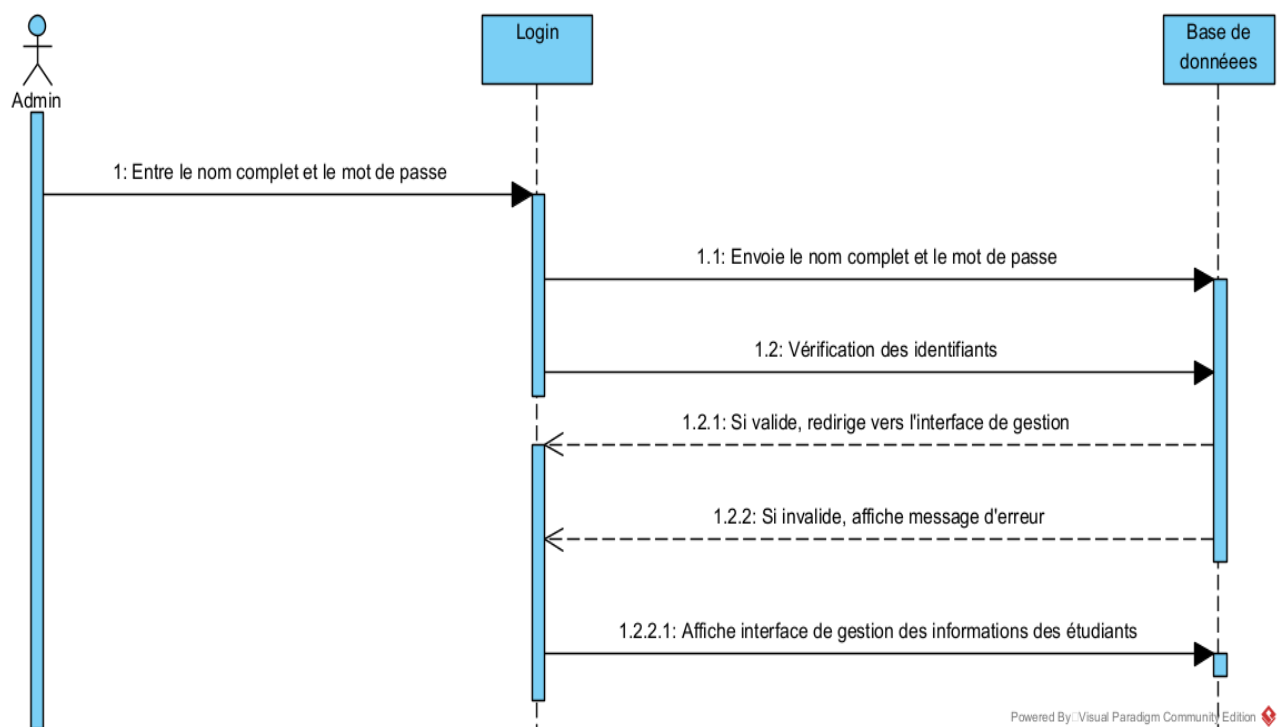


Figure : Diagramme de séquence Authentification

3.2 Ajouter un étudiant :

➤ Flux des événements :

1. Entrez les infos
 - 1.1 Valide les informations
 - 1.2 Appel à `btnAjoutActionPerformed()`
 - 1.3 Envoie les données à la base de données
 - 1.3.1 **Système** Confirmation ajoutée
 - 1.3.2 **Système** Mise à jour de l'interface

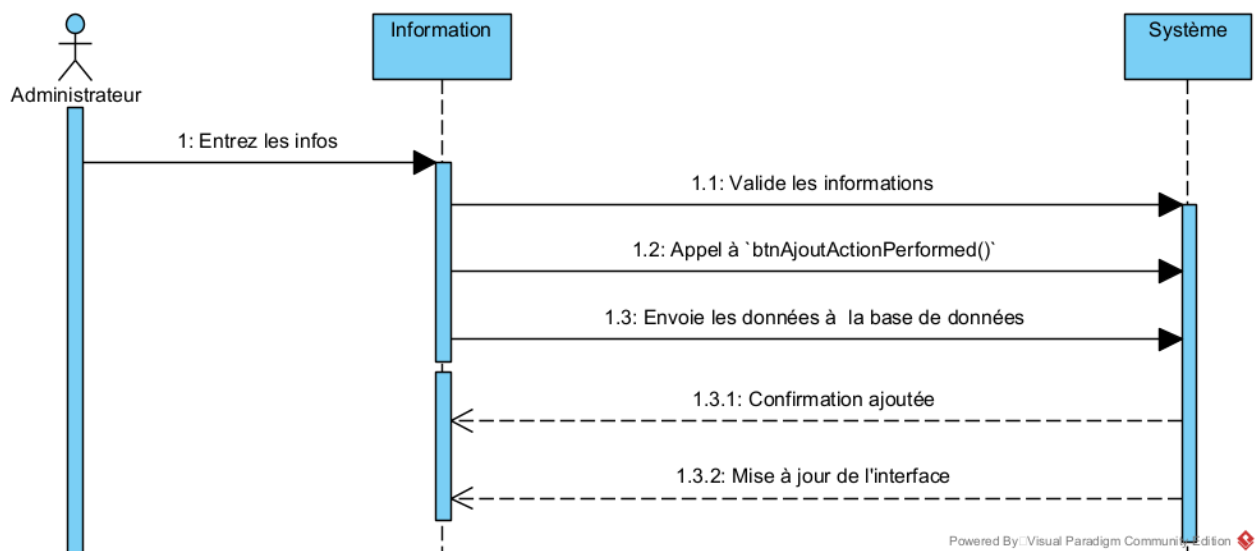


Figure : Diagramme de séquence d'Ajouter étudiant

3.3 Afficher la liste des étudiants :

➤ Flux des événements :

1. Demander la liste des étudiants
 - 1.1 getAllEtudiants()
 - 1.1.1 Créer un objet Etudiant
 - 1.1.2 **Système** Retourner un ArrayList<Etudiant>
 - 1.1.2.1 **Information** Afficher la liste des étudiants

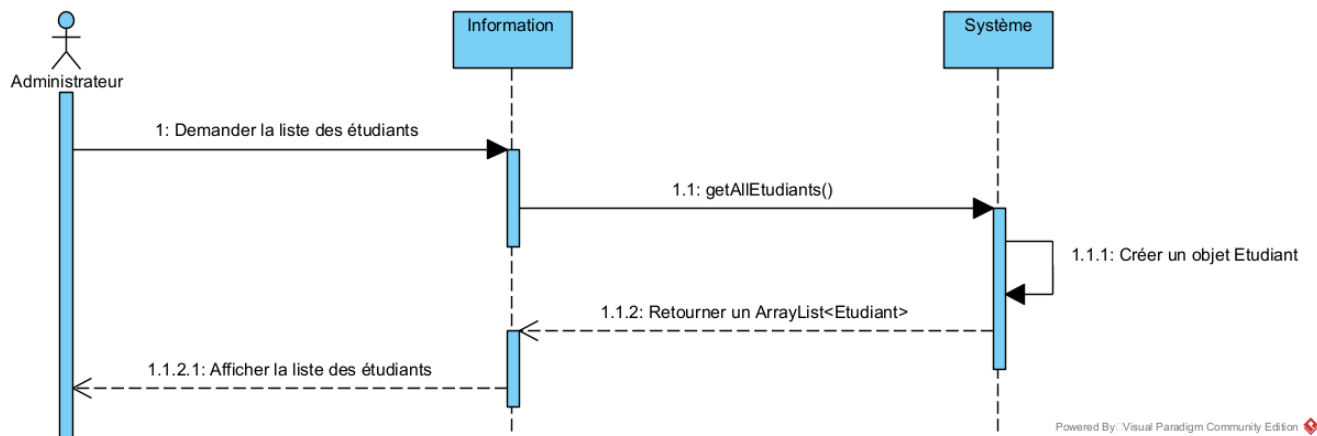


Figure : Diagramme de séquence d’Afficher la liste des étudiants

3.4 Sélectionner un étudiant pour le modification suppression :

➤ Flux des événements :

1. Demander la liste des étudiants
 - 1.1 getAllEtudiants()
 - 1.1.1 **Système** Retourner la liste des étudiants
 - 1.1.1.1 **Information** Afficher la liste
2. Sélectionner un étudiant
 - 2.1 **Information** Afficher les détails de l'étudiant sélectionné

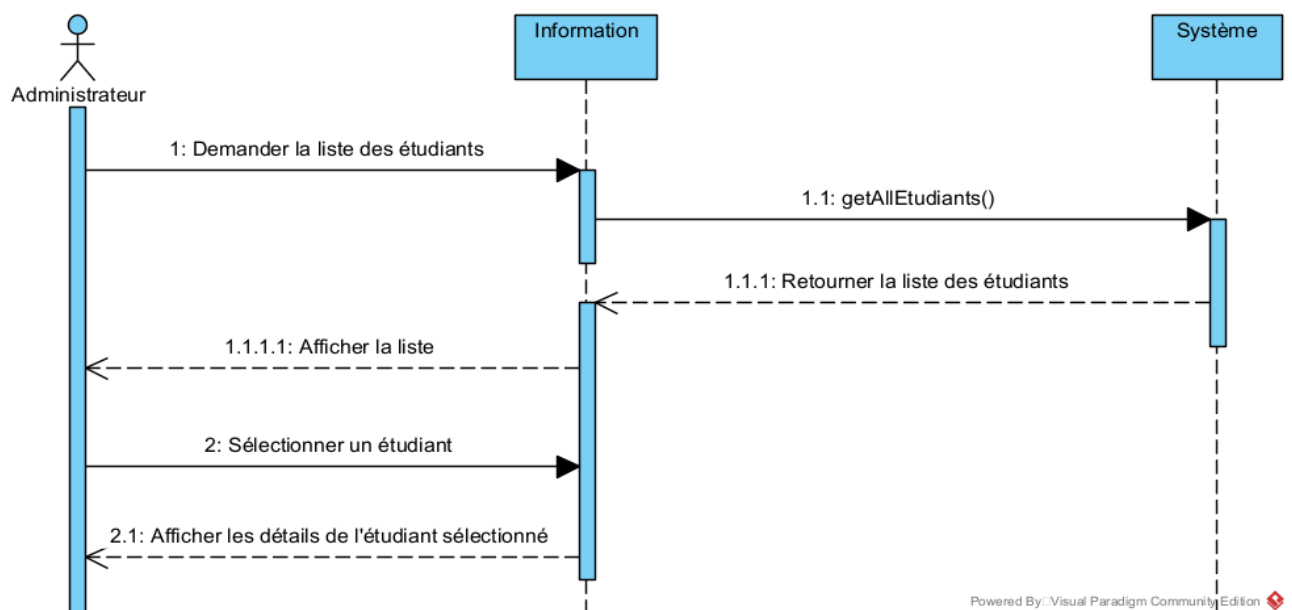


Figure : Diagramme de séquence de Sélectionner un étudiant pour le modification suppression

3.5 Modifier un étudiant :

➤ Flux des événements :

1. Sélectionne l'étudiant
 - 1.1 Affiche les informations
 - 1.2 Modifie les informations
 - 1.3 Appel à `btnModMouseClicked()`
 - 1.4 Appel à `btnModMouseClicked()`
 - 1.4.1 **Système** Confirmation modifiée

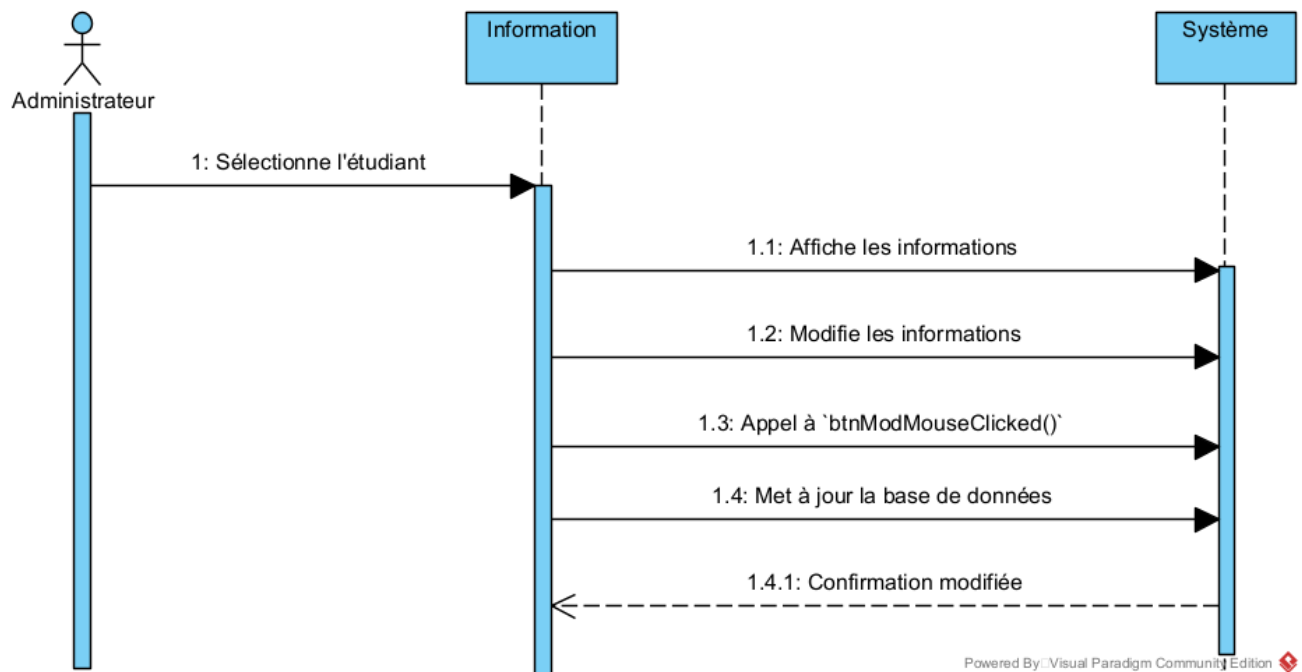


Figure : Diagramme de séquence de Modifier un étudiant

3.6 Supprimer un étudiant :

➤ Flux des événements :

1. Sélectionne l'étudiant
 - 1.1 Affiche les informations
 - 1.2 Confirme la suppression

- 1.3 Appel à `btnSupActionPerformed()`
- 1.4 Supprime les données dans la BD
- 1.5 **Système** Confirmation de suppression
- 1.6 **Système** Mise à jour de l'interface

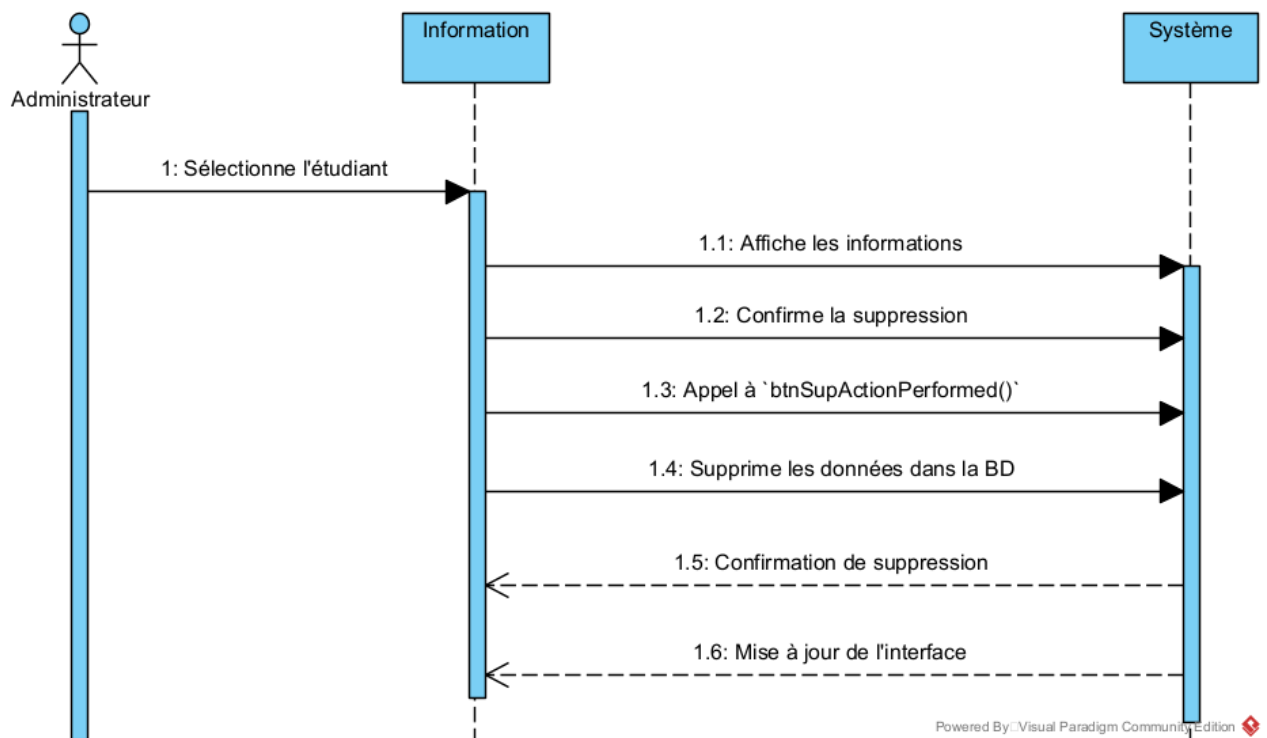


Figure : Diagramme de séquence de Supprimer un étudiant

4. Diagrammes d'activité :

4.1 Authentification :

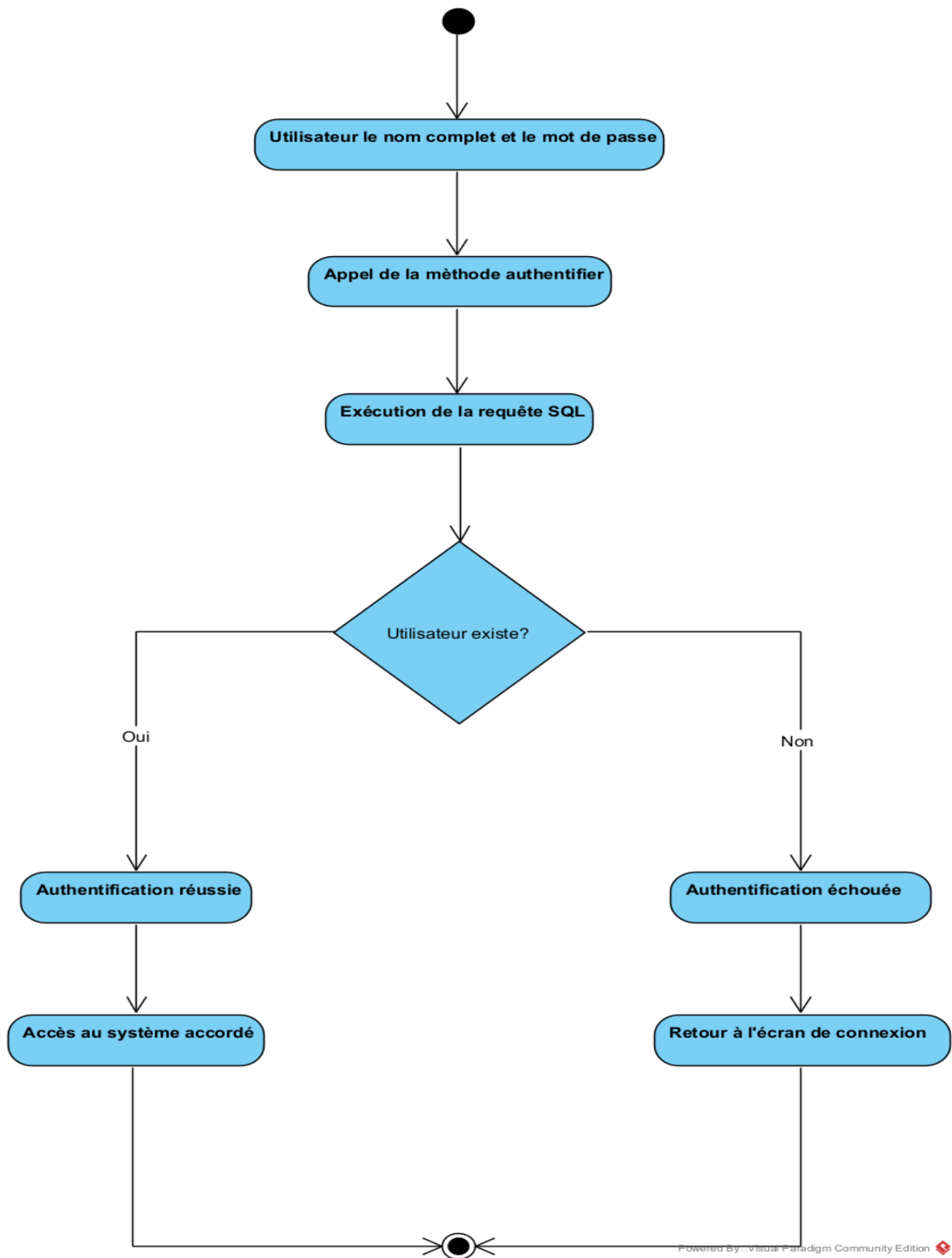


Figure : Diagramme d'activité Authentification

4.2 Ajouter un étudiant :

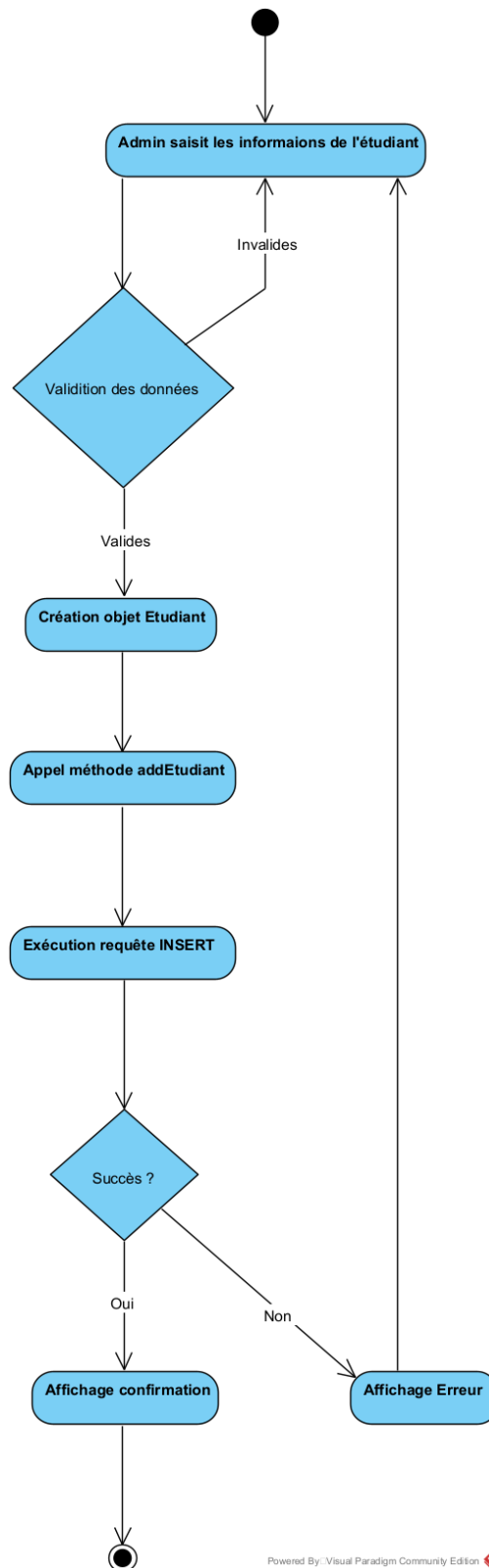


Figure : Diagramme d'activité d'Ajouter étudiant

4.3 Afficher la liste des étudiants :

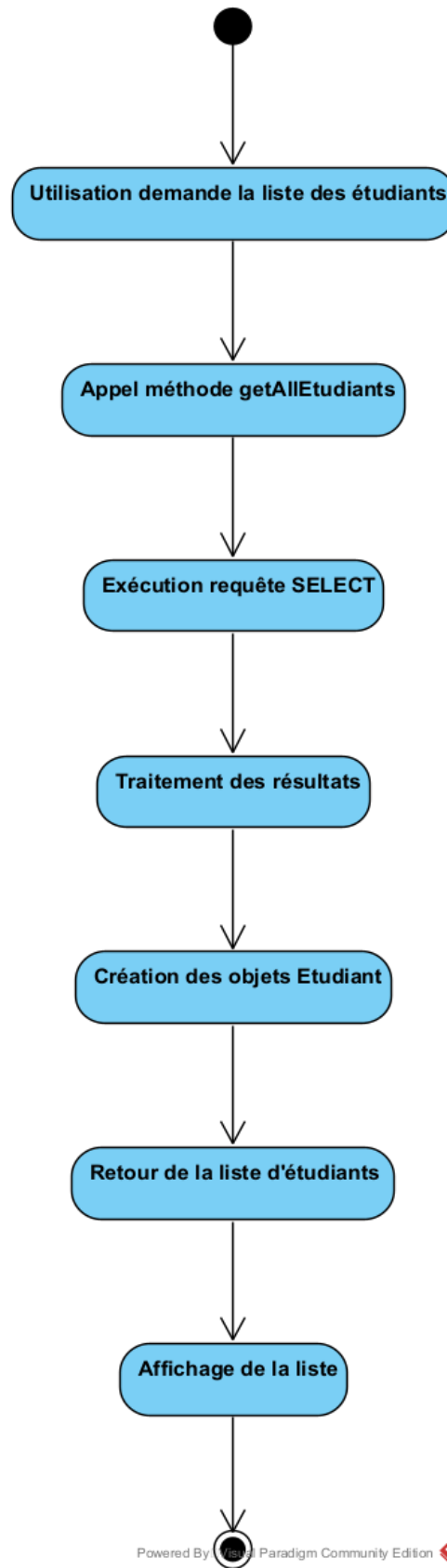


Figure : Diagramme d'activité d'Afficher la liste des étudiants

4.4 Sélectionner un étudiant pour le modification suppression :

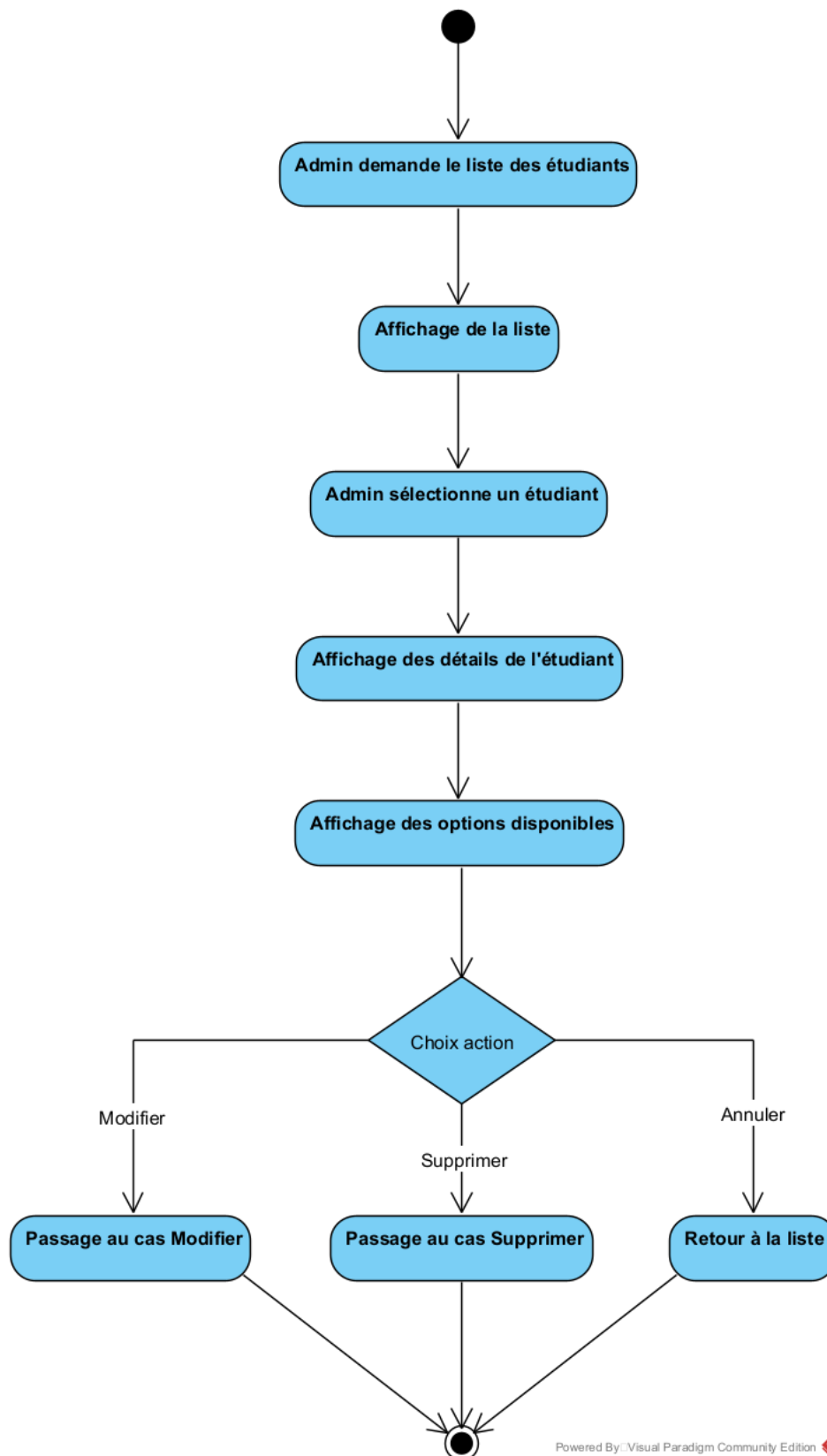


Figure : Diagramme d'activité de Sélectionner un étudiant pour le modification suppression

4.5 Modifier un étudiant :

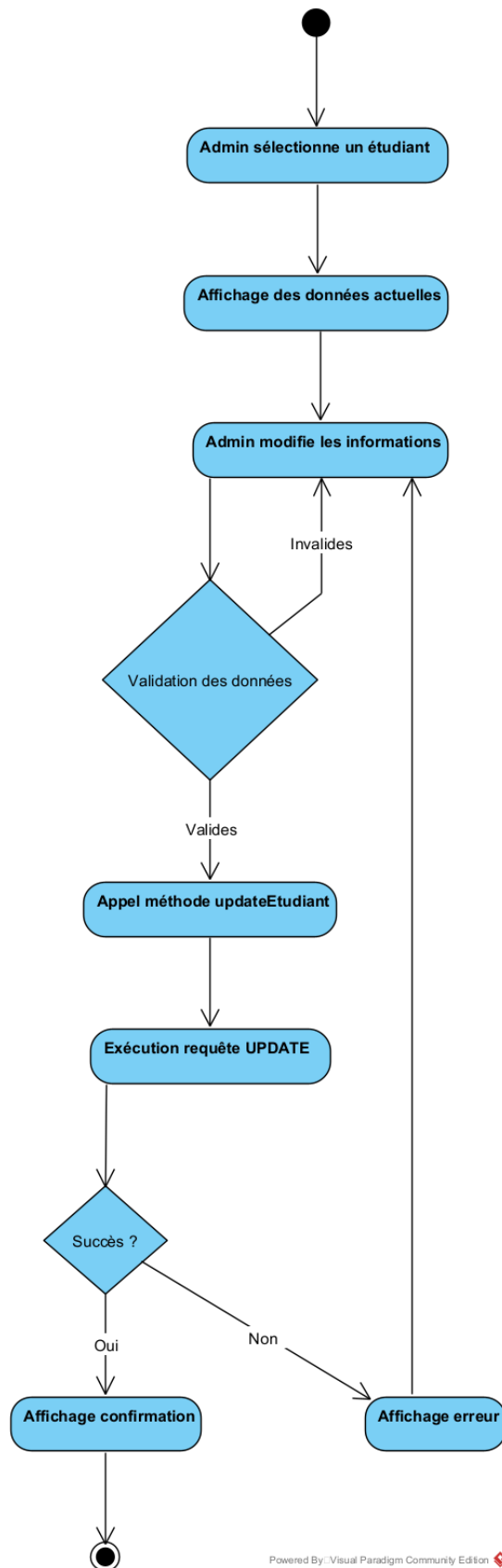


Figure : Diagramme d'activité de Modifier un étudiant

4.6 Supprimer un étudiant :

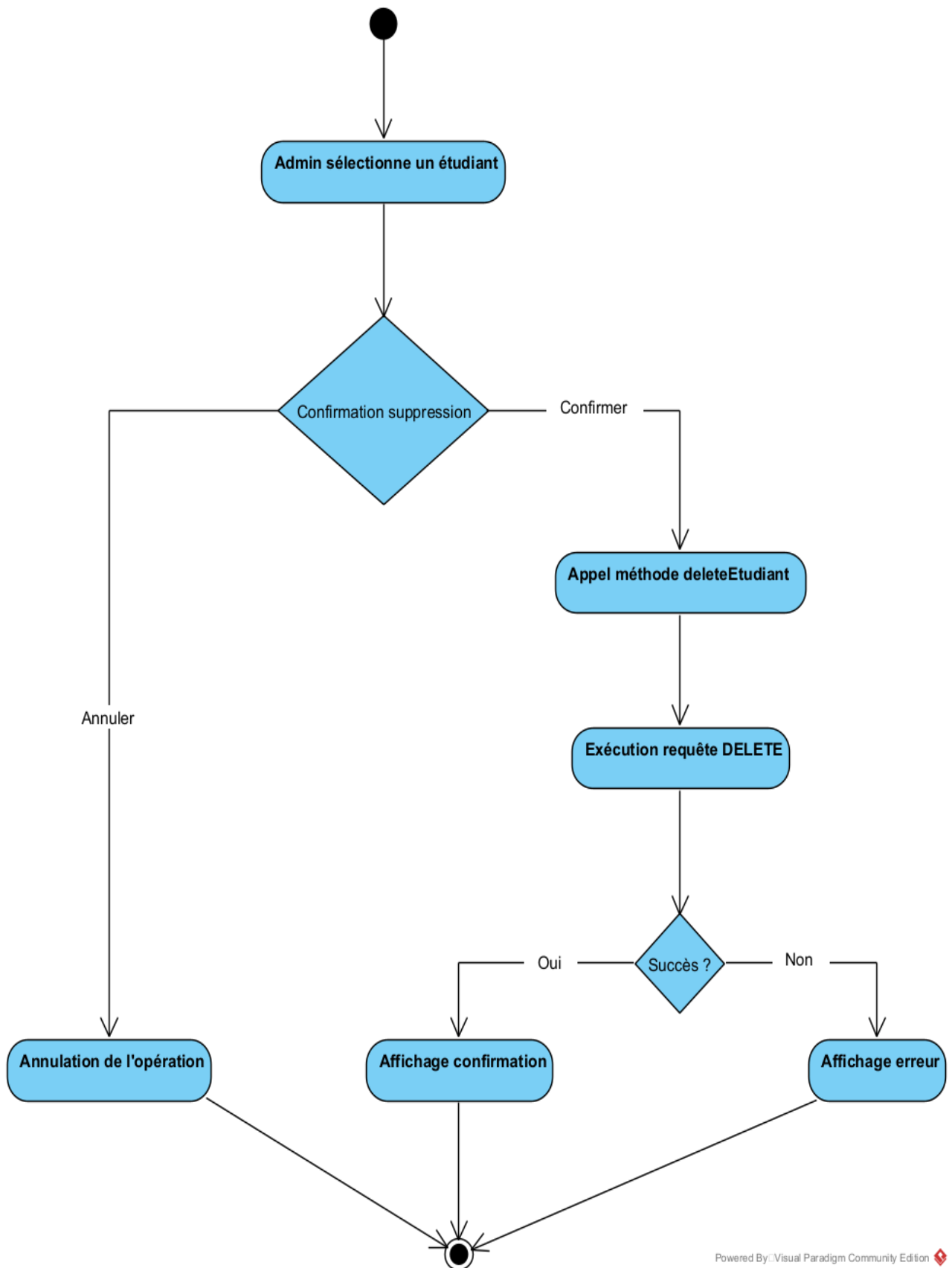


Figure : Diagramme d'activité de Supprimer un étudiant

5. Diagramme de classe :

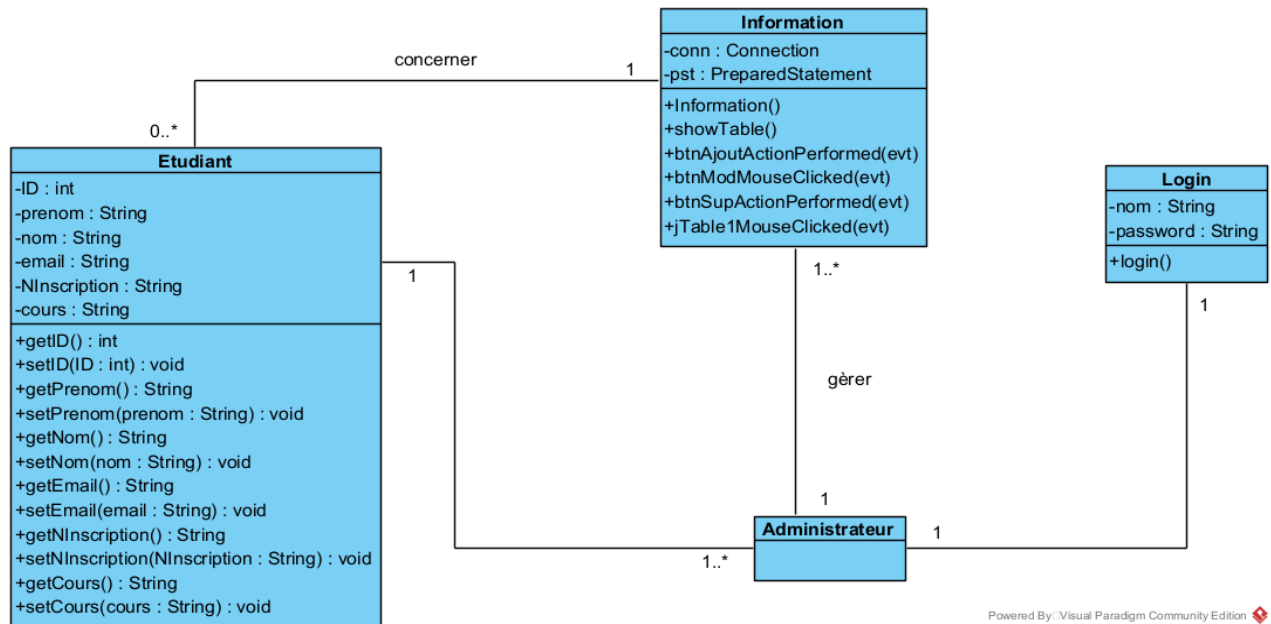
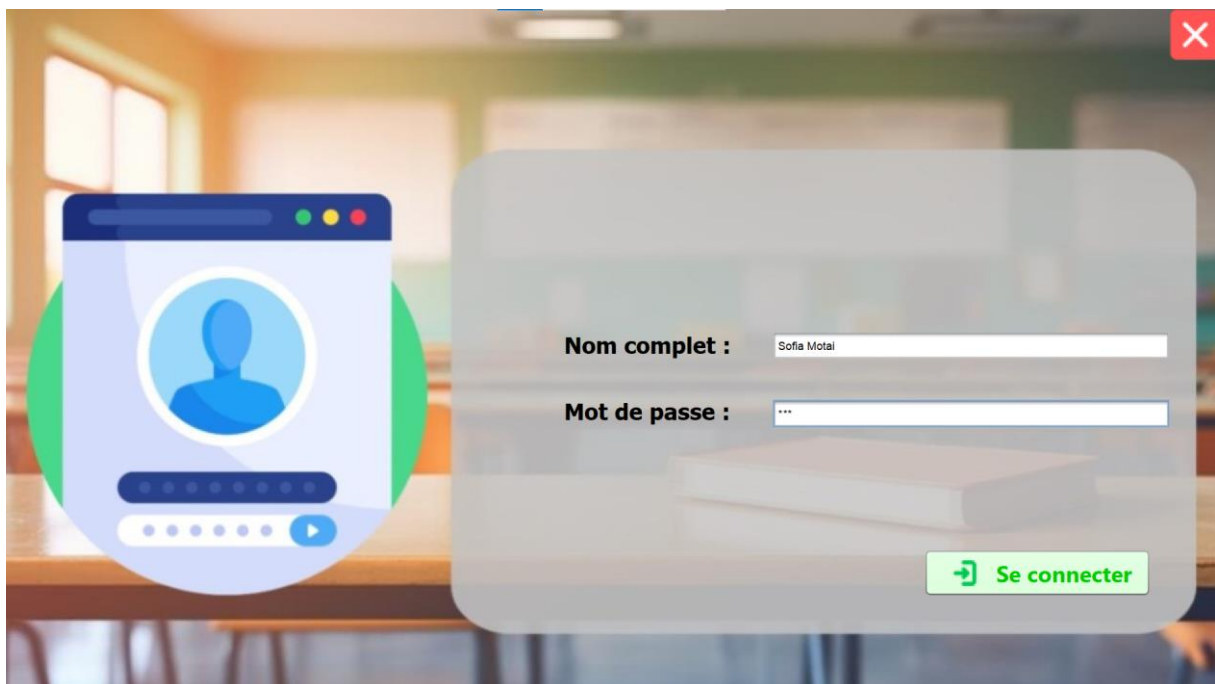


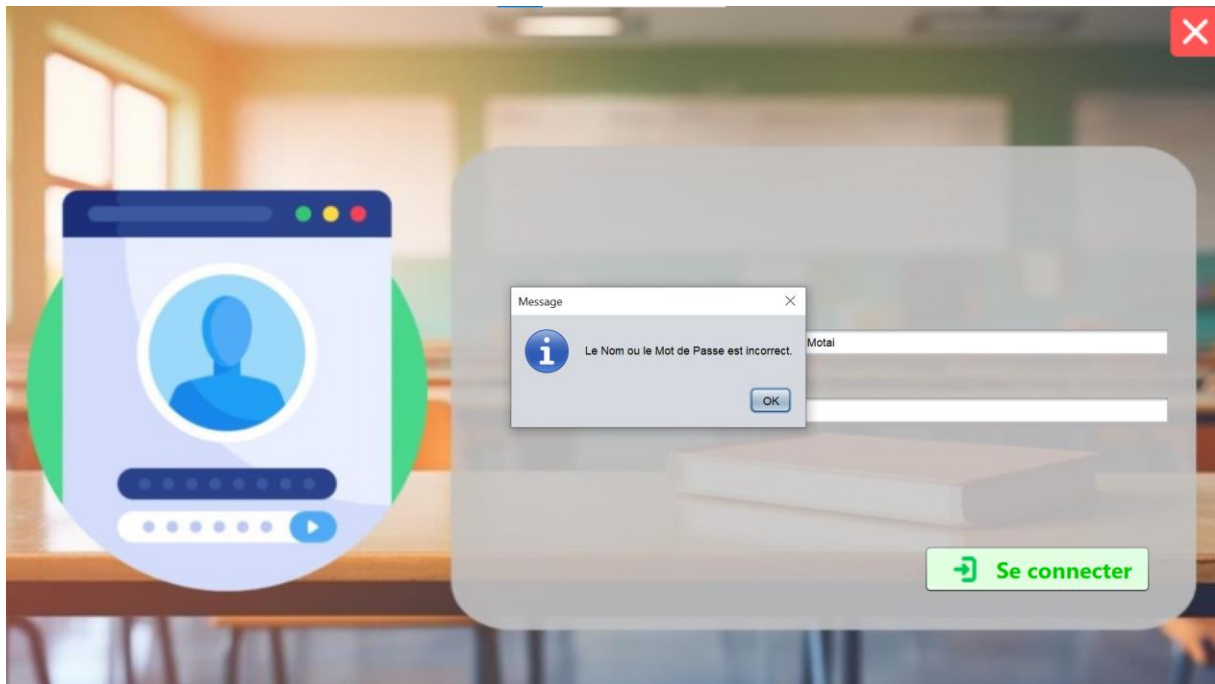
Figure : Diagramme de classe de gestion des  tudiants

III. R alisation :

1. Page de Connexion de l'Administrateur :



- L'administrateur saisit son **nom complet** et son **mot de passe**.
- Un bouton "Se connecter" est bien visible et permet de lancer le processus.



- Si les informations sont incorrectes, une **boîte de dialogue** s'affiche : **"Le Nom ou le Mot de Passe est incorrect."**
- Cela montre une bonne gestion des erreurs d'authentification.

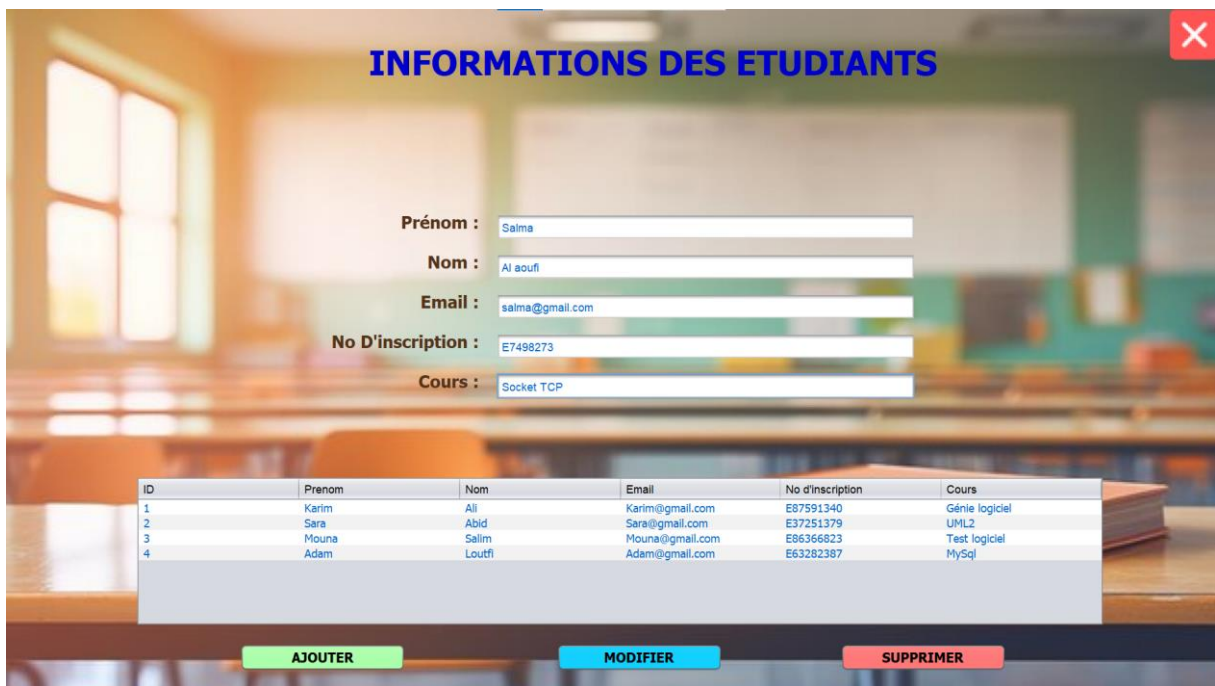
2. Interface Principale - Gestion des Étudiants :

The image shows the main interface for managing students, titled 'INFORMATIONS DES ETUDIANTS'. It features a form with the following fields: 'Prénom', 'Nom', 'Email', 'No D'inscription', and 'Cours'. Below the form is a table listing student information. At the bottom, there are three buttons: 'AJOUTER' (green), 'MODIFIER' (blue), and 'SUPPRIMER' (red).

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySql

- Une fois connecté, l'administrateur accède à l'interface des étudiants.
- Les champs de saisie permettent d'entrer les infos suivantes :
 - Prénom
 - Nom
 - Email
 - N° d'inscription
 - Cours
- Les données des étudiants sont affichées dans une **JTable** en dessous.
- Boutons :
 - AJOUTER (vert clair)
 - MODIFIER (bleu)
 - SUPPRIMER (rouge clair)

3. Ajout d'un Étudiant :



INFORMATIONS DES ETUDIANTS

Prénom :

Nom :

Email :

No D'inscription :

Cours :

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySql

AJOUTER MODIFIER SUPPRIMER

- L'administrateur saisit les informations d'un nouvel étudiant.

INFORMATIONS DES ETUDIANTS

Prénom :

Nom :

Email :

No D'inscription :

Cours :

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySQL
67	Salma	Al aoufi	salma@gmail.com	E7498273	Socket TCP

- Une fois le bouton **AJOUTER** cliqué, une boîte de dialogue confirme : *"Data ajouter avec succès !"*
- Le nouvel étudiant apparaît immédiatement dans la table avec un nouvel ID.
- Rafraîchit le tableau.

4. Modification :

INFORMATIONS DES ETUDIANTS

Prénom :

Nom :

Email :

No D'inscription :

Cours :

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySQL
67	Salma	Al aoufi	salma@gmail.com	E7498273	Socket TCP

- Sélectionner une ligne du tableau
- Les données apparaissent dans le formulaire
- On a modifié le No D'inscription de E7498273 à E74982791

The screenshot shows a web application titled "INFORMATIONS DES ETUDIANTS". It features a form with fields for "Prénom", "Nom", "Email", "No D'inscription", and "Cours". A modal dialog box is open, displaying the message "Data modifier avec succès !" (Data modified successfully!) with an "OK" button. Below the form is a table of student data. At the bottom, there are three buttons: "AJOUTER" (green), "MODIFIER" (blue), and "SUPPRIMER" (red).

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySql
5	Salma	Ali	Salma@gmail.com	E74982791	Scalad 100

- Une fois le bouton **MODIFIER** cliqué, une boîte de dialogue confirme :
"Data modifier avec succès !"
- Mise à jour en base via JDBC.

5. Suppression :

The screenshot shows the same web application, but the modal dialog box now displays the message "Enregistrement supprimé !" (Record deleted!). The table below the form shows the same data as the previous screenshot, with the student "Salma" (ID 5) still present.

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySql
5	Salma	Ali	Salma@gmail.com	E74982791	Scalad 100

- Sélectionner une ligne pour la supprimer.
- Une fois le bouton **SUPPRIMER** cliqué, une boîte de dialogue confirme :
"Enregistrement supprimé !"

ID	Prenom	Nom	Email	No d'inscription	Cours
1	Karim	Ali	Karim@gmail.com	E87591340	Génie logiciel
2	Sara	Abid	Sara@gmail.com	E37251379	UML2
3	Mouna	Salim	Mouna@gmail.com	E86366823	Test logiciel
4	Adam	Loutfi	Adam@gmail.com	E63282387	MySql

- Rafraîchissement du tableau.

6. Test unitaire JUnit :

6.1 ConnexionTest :

Test	But	Résultat attendu
testConnexionSucces()	Tester que la méthode <i>Connexion.getCon()</i> fonctionne correctement	La connexion doit réussir (non nulle)
testConnexionEchec()	Vérifier qu'une tentative de connexion avec de mauvaises infos échoue	La connexion doit être nulle

6.2 DAOTest :

- Structure **du test** : Le code utilise le framework JUnit avec les annotations **@Before**, **@After** et **@Test** pour organiser les tests.
- Méthode **setUp (@Before)** : Cette méthode s'exécute avant chaque test et :
 - Initialise un objet DAO
 - Crée un étudiant test nommé "Rania Karam"

- Ajoute cet étudiant à la base de données via la méthode `dao.addEtudiant()`
- Méthode **tearDown (@After)** : Elle nettoie après chaque test en supprimant tous les étudiants nommés "Rania Karam" de la base de données.
- Tests **implémentés** :
 - **testAddEtudiant** : Vérifie qu'un étudiant peut être ajouté correctement
 - **testUpdateEtudiant** : Vérifie la modification des informations d'un étudiant existant
 - **testDeleteEtudiant** : Vérifie la suppression d'un étudiant
 - **testGetAllEtudiants** : Vérifie que la méthode retourne une liste d'étudiants non vide

6.3 EtudiantTest :

- Structure : Ce test est plus simple que le précédent, utilisant uniquement l'annotation `@Test` sans `@Before` ou `@After`.
- Tests **implémentés** :
 - **testConstructeurParDéfautEtSettersEtGetters** : Vérifie le fonctionnement du constructeur par défaut et de tous les getters/setters
 - **testConstructeurAvecTousLesChamps** : Vérifie le constructeur qui initialise tous les attributs
 - **testConstructeurSansID** : Vérifie un constructeur intermédiaire qui n'initialise pas l'ID
- Classe **Etudiant** :
 - Elle possède 6 attributs : ID, prénom, nom, email, numéro d'inscription et cours
 - Elle dispose de 3 constructeurs différents :
 - Un constructeur par défaut sans paramètres
 - Un constructeur complet avec tous les champs
 - Un constructeur sans l'ID (probablement utilisé lors de la création d'un nouvel étudiant avant attribution d'un ID par la base de données)
 - Elle dispose de getters et setters pour tous ses attributs

6.4 LoginTest :

- Structure : Comme pour le test de la classe `Etudiant`, c'est un test JUnit simple avec uniquement l'annotation `@Test`.
- Tests **implémentés** :

- **testAuthentifierAvecIdentifiantsValides** : Vérifie que l'authentification fonctionne correctement avec des identifiants valides
- **testAuthentifierAvecIdentifiantsInvalides** : Vérifie que l'authentification échoue avec des identifiants invalides
- Classe **Login** : Ce test nous permet de déduire que :
 - La classe `Login` dispose d'une méthode ***authentifier (String username, String password)*** qui retourne un booléen
 - Cette méthode retourne ***true*** si les identifiants sont valides et ***false*** sinon
 - Il semble y avoir un utilisateur prédéfini avec le nom "Sofia Motai" et le mot de passe "123"

IV. Conclusion :

Le projet "**Gestion des Informations des Étudiants**" représente une application concrète de la méthodologie **RUP (Rational Unified Process)** dans le cadre du développement logiciel orienté objet. À travers une approche structurée et itérative, nous avons pu modéliser, concevoir, implémenter et tester une application Java robuste et conviviale.

L'analyse approfondie des besoins a permis de bien cadrer les fonctionnalités clés telles que l'ajout, la modification, la suppression et l'affichage des étudiants, tout en intégrant des exigences de qualité et de performance. L'utilisation des **cas d'utilisation, diagrammes UML (séquences, activités, classes)** a renforcé la clarté de la conception tout en assurant une traçabilité complète entre les exigences et leur mise en œuvre.

La phase de **réalisation** a mis en avant une application Java Swing couplée à une base de données, avec une séparation claire des responsabilités (interface, logique métier, accès aux données). L'intégration des **tests unitaires via JUnit** a permis de valider la fiabilité de chaque composant, tout en garantissant la maintenabilité du code sur le long terme.

En somme, ce projet démontre non seulement une maîtrise des **bonnes pratiques de développement Java**, mais aussi une compréhension solide des principes d'ingénierie logicielle. La méthodologie RUP, par sa rigueur et son organisation, a permis de piloter efficacement le développement, de la phase d'analyse jusqu'aux tests finaux, aboutissant à une application fonctionnelle, évolutive et orientée qualité.